

WildGraph: Graph-Based Animal Classification System

PRESENTED BY
EXECUTIONERS



Objectives of Project

- Telling apart hundreds of look-alike animal species is genuinely hard, even for experts
- WildGraph builds a map of animal images where visually similar species are connected, like a web of resemblances
- Instead of guessing each species in isolation, it understands how animals relate to each other
- Works with 398 species and subspecies, many of which are near-identical variations of the same animal
- Similarity search: show it a photo, it finds the most visually alike animals in the dataset
- Classification: it identifies an unknown animal by looking at what its closest visual neighbors tend to be



The Excruciating extraction of Image data

Having to get rid or mask the background

Most of the pre-existent data set included a lot of background noise...literally, from trees to even large bodies of water

The approach?

- by figuring out which parts of an image are visually "interesting" to the human eye. It assigns each pixel a score based on how much it stands out from its surroundings, from Sharpness, brightness or texture, everything bland, anything repetitive scoring low

The result?

A heat map where the subject pops and the background fades

Detecting the Edges and colour

```
... object to mirror  
... mirror_mod.mirror_object
```

```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
... selection at the end -add  
... mirror_ob.select= 1  
... mirror_ob.select=1  
... context.scene.objects.active  
... ("Selected" + str(modifier_name))  
... mirror_ob.select = 0  
... bpy.context.selected_objects  
... data.objects[one.name].select  
... print("please select exactly one object")
```

```
... OPERATOR CLASSES -----
```

```
... types.Operator):  
    ... X mirror to the selected  
    ... object.mirror_mirror_x"  
    ... mirror X"
```

```
... ):  
    ... object is not
```



Taking that “Saliency” image and splitting into multiple pieces

Now while extracting features from the whole image would have seemed like the fastest thing to do it wasn't necessarily the wisest, we need to be as accurate as possible

Solution?

- The “Saliency” image needed to be broken down into segments so features could be extracted piece by piece rather than from the whole image at once. It would get divided into a grid of sections, each section becoming its own cropped sub-image called a segment, each keeping track of its local binary pattern, for integrity of course

These regions then get bundled and thrown into a regional batch, essentially a structured collection ready to be passed down the pipeline

What the Saliency filter actually does

Before edges could be detected, the image needed to be gets converted to grayscale first, using the proper luminance formula so the brightness values actually reflect how human eyes perceive colour rather than just plainly averaging out

formula in question : $(0.299 * \text{Red} + 0.587 * \text{Green} + 0.114 * \text{Blue})$;

- The saliency filter then looks at every pixel and asks a simple question: how different is this pixel from everything around it? It does this by comparing local colour and brightness against the broader regional average. A pixel sitting in the middle of a flat grassy field looks basically identical to its neighbours, low contrast, low saliency, not interesting. A pixel sitting on the edge of a zebra stripe next to a completely different colour? That one screams.

The difference gets assigned a score, high score means the pixel stands out, it is probably part of the subject. Low score means it blends in, it is probably background. Stack all those scores across the whole image and you get a heat map, bright where the animal is, dark where the trees and water are

What the Sobel filter actually does (continued)

Breaking the image into pieces

The 16-region grid was the first attempt. Clean, simple, and completely wrong. It didn't care about the animal, it just cuts wherever it felt like

It basically blew up in our faces

How did we Improve?



- SLIC segmentation ; Instead of forcing the image into fixed squares, we grew regions around "seed" points placed across the image in a grid, Each seed would then pull in the pixels closest to it, not just spatially but in colour too, the pixel would have to be both nearby and similar to join a group

Once pixels were assigned, the seeds would move to the true centre of their group. Then it would repeat

The Result?

A set of blobs that follow the animal's fur lines, its edges and actual anatomy

Graph construction

```
...mirror_mod.mirror_object = mirror_mod.mirror_object
...operation == "MIRROR_X":
...    mirror_mod.use_x = True
...    mirror_mod.use_y = False
...    mirror_mod.use_z = False
...    operation == "MIRROR_Y":
...    mirror_mod.use_x = False
...    mirror_mod.use_y = True
...    mirror_mod.use_z = False
...    operation == "MIRROR_Z":
...    mirror_mod.use_x = False
...    mirror_mod.use_y = False
...    mirror_mod.use_z = True
```

```
...selection at the end -add
...mirror_ob.select= 1
...modifier_ob.select=1
...context.scene.objects.active
...("Selected" + str(modifier_ob.name))
...mirror_ob.select = 0
...= bpy.context.selected_objects[0]
...data.objects[one.name].select
...print("please select exactly one object")
```

--- OPERATOR CLASSES ---

```
...types.Operator):
...    X mirror to the selected
...    object.mirror_mirror_x"
...    mirror X"
```

```
...def execute(self, context):
...    if context.object is not None:
```

Graph construction, the skeleton of the animal

Now that we have bobs, essentially the anatomy of the animal, the question is, of which of these block are of closer proximity to each other?

How we find that out?

- We scan every pixel in the image and check its' right and bottom neighbour, If two neighbouring pixels belong to different blobs, those two blobs get connected by an edge...Boom

The Result?

A Region Adjacency Graph. Each node a blob, each edge meaning two blobs share a physical border in the image

Classification



(the good part)
\(^o^)/

Classification, making the ACTUAL decision

The Graph exists of course, but now, what can we do with it?

Comparing two graphs at first got to messy and heavy, so what did we do?, we summarised each one into a sort of signature, short list of numbers that captures what the graph looks like

What does it do?

- It tracks two things mainly
 - a) What types of nodes exist
 - b) How connected they are on average



Those signatures would go into a k-NN classifier; When an unknown animal comes in, its signature gets compared against every stored signature in the training data